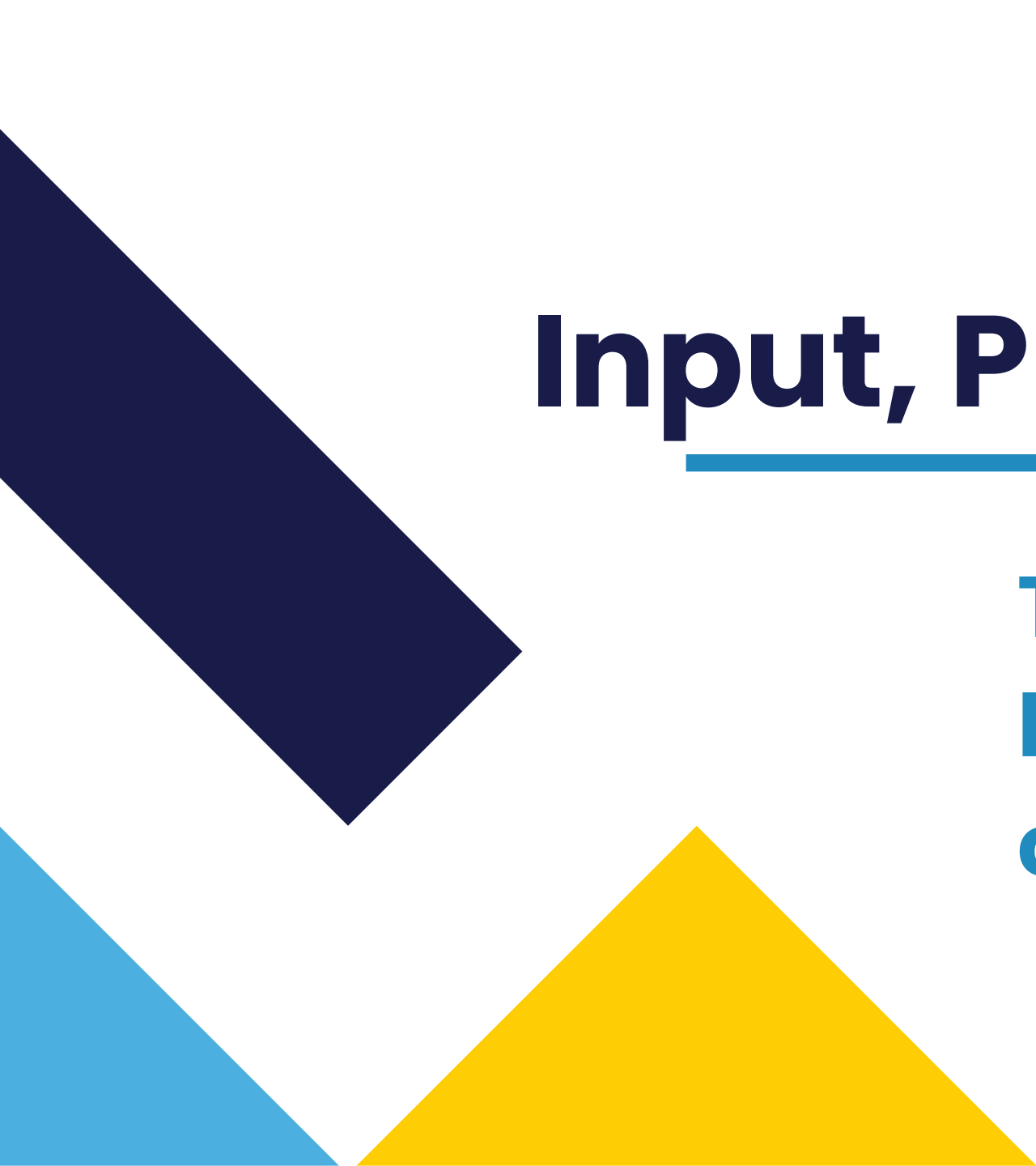




Input, Process, Output

**The basic
breakdown of any
computation!**

Input, Process, and Output

Most programs can be broken into three main parts:
input, process, and output.

Input is information that enters the program,
process is what the program does with that
information, and output is the result produced.

This is called the **Input-Process-Output (IPO)
Model.**

Planning with IPO Charts

An **IPO Chart** identifies what information a program needs, what it will do with that information, and what result it will produce.

IPO charts help organize a program before any code is written.

Each section describes the purpose of that stage rather than the exact Python syntax.

INPUT	PROCESS	OUTPUT
User enters their name	Store the name	Display a greeting

Input – User Input

Input is information provided to a program so it can perform a task.

Programs can receive input from many sources, including keyboards, sensors, cameras, and files.

In Python, `input()` is used to collect text entered by the user in the terminal.

The program waits at `input()` until the user types a response and presses **Enter**.

```
input("What is your name? ")
```

OUTPUT

What is your name? *Mr. Forsyth*

Prompt

Typed by user

Input – Input Prompts

The string inside `input()` is called the **prompt**.

It is displayed before Python waits for the user's response.

A clear prompt tells the user exactly what information they should enter.

```
input("Enter your favourite colour: ")
```

OUTPUT

Enter your favourite colour: *Orange*

Prompt

Typed by user

Input – Storing User Input

The value returned by `input()` can be assigned to a variable.

This allows the program to remember the user's response and use it later.

The variable stores whatever text the user entered.

```
name = input("What is your name? ")  
  
print(name)
```

OUTPUT

```
What is your name? Mr. Forsyth  
  
Mr. Forsyth
```

Process – Using Input in Your Program

A **process** is an action performed on data, such as a calculation, modification, or decision.

After receiving input, a program can process that information and then display a result.

The example code in this slide includes some concepts covered later in these slides.

```
number = input("Enter a number: ")
```

Input

```
number = int(number)  
doubled = number * 2
```

Process

```
print(f"Double your number is {doubled}.")
```

Output

Process – User Input Is a String

The `input()` function always returns a **string**, even when the user types a number.

This means `"5"` and `5` are different values: `"5"` is text, while `5` is an integer.

If Python tries to use the string `"5"` in a mathematical operation, it can cause an error because it is not being treated as a number.

```
name = input("What is your name? ")  
  
print(name)
```

OUTPUT

```
Enter a number: 5
```

```
TypeError: can only concatenate str (not "int") to str
```

Process – Casting Input to **int** or **float**

Casting converts a value from one datatype to another.

int() converts a string into a whole number.

float() converts a string into a decimal number.

This allows numeric user input to be used in calculations.

```
whole_number = input("Enter a whole number: ")
whole_number = int(whole_number)
print(whole_number)

decimal_number = input("Enter a decimal number: ")
decimal_number = float(decimal_number)
print(decimal_number * 2)
```

OUTPUT

```
Enter a whole number: 5
5
Enter a decimal number: 2.5
5.0
```

Output with `print()`

Output is information produced by a program.

In Python, `print()` is commonly used to display output in the terminal.

It can display strings, variables, numbers, and other values.

```
name = "Alex"  
score = 10  
  
print(name)  
print(score)
```

OUTPUT

Alex

10

Output – Combining Strings with Concatenation

Concatenation means joining strings together using **+**.

Because concatenation joins strings, other datatypes must first be converted using **str()**.

This works well for simple messages, but larger statements can become difficult to read.

```
name = "Alex"  
score = 10  
  
print("Player: " + name + " Score: " + str(score))
```

OUTPUT

Player: Alex Score: 10

Output – Combining Output with Commas

Multiple values can be placed inside `print()` by separating them with commas.

Python automatically adds a space between the values and can display different datatypes together.

This is a simple way to combine text and variables.

The downside to using commas is that it always adds a space between your values.

```
name = "Alex"  
score = 10  
  
print("Player:", name, "Score:", score)
```

OUTPUT

Player: Alex Score: 10

Output – Formatted Strings

A **formatted string**, or **f-string**, allows variables and expressions to be inserted directly into a string.

Add **f** before the opening quotation mark and place values inside curly brackets **{}**.

F-strings are usually the clearest way to create output containing several variables.

They can also include numbers without converting them to strings first.

```
name = "Alex"  
score = 10  
  
print(f"Player: {name} | Score: {score}")
```

OUTPUT

```
Player: Alex | Score: 10
```

Formatting Text – Escape Characters

An **escape character** changes how Python interprets the character that follows it.

Python uses the backslash `\` as its escape character.

*(Make sure you don't get confused with the forward slash character `/` which is **not an escape character**).*

This allows special characters, such as quotation marks, to be placed inside strings without ending the string early.

```
message = "Alex said, \"Hello!\""
print(message)
```



Notice that the escaped quotation marks change to orange!

OUTPUT

```
Alex said, "Hello!"
```

Formatting Text – The Newline Character `\n`

`\n` creates a new line inside a string.

It can be used in normal strings, formatted strings, or input prompts.

This is useful when output needs to be split across multiple lines.

```
print("First Line\nSecond Line")
```

OUTPUT

```
First Line  
Second Line
```

Pro Tip: Using `\n` in an Input Prompt

A newline can also be placed inside the prompt used by `input()`.

This moves the user's typing position onto the next line.

It can make longer prompts easier to read.

```
name = input("Enter your name:\n")  
  
print(f"Hello, {name}!")
```

OUTPUT

```
Enter your name:  
Mr. Forsyth  
Hello, Mr. Forsyth!
```

IPO Wrap Up

Python programs commonly follow the **Input-Process-Output (IPO)** model.

input() collects information from the user and always returns that information as a string, so numeric input may need to be cast with **int()** or **float()**.

print() displays output, and values can be combined using commas, concatenation, or **f-strings**, with f-strings usually providing the clearest formatting.

Escape sequences such as **\n** and **\"** can also be used to control how strings are displayed.